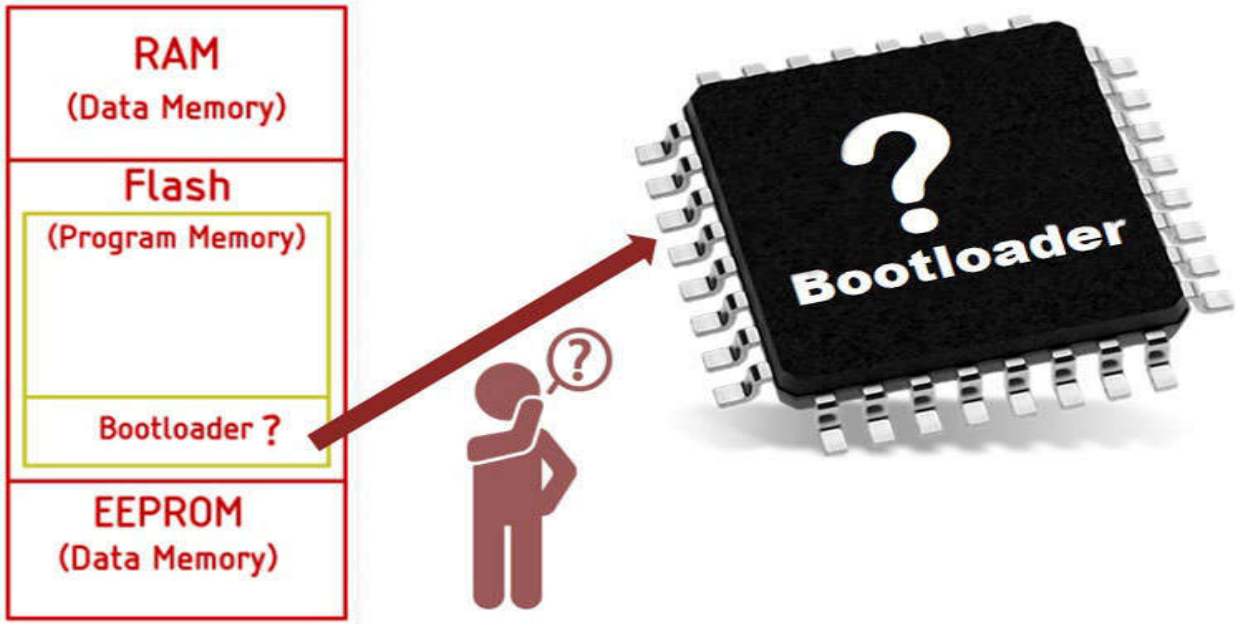
Chuyên đề

# BOOTLOADER

**TS. Đào Việt Hùng**
Khoa Điện tử
Trường Điện – Điện tử
Đại học Bách Khoa Hà Nội
Email: hunget.bk@gmail.com

# Nội dung

- Tổng quan về Bootloader
- Thiết kế và phân vùng bộ nhớ
- Bootloader trên chip STM32F4



# 1 – Tổng quan về Bootloader

- Định nghĩa
- Cơ chế hoạt động
- Phân loại
- Mục đích sử dụng

# 1.1 – Định nghĩa

- Bootloader là công cụ mà các nhà sản xuất hoặc lập trình viên sinh ra, được bật lên khi cần để điều hướng các Firmware dùng đến khi khởi động
- Bản chất, Bootloader là một chương trình phần mềm siêu nhỏ cư trú tại địa chỉ thực thi lệnh đầu tiên (nếu được sử dụng) sau khi vi điều khiển khởi động hoặc Reset

# 1.2 – Cơ chế hoạt động

- Tuỳ thuộc vào nhân của dòng vi điều khiển sử dụng, người lập trình có những cách thức khác nhau để làm việc với chế độ Bootloader
- Tuy nhiên, mọi hệ thống Bootloader đều vận hành dựa trên 03 bước cốt lõi
  - Bước 1: Thiết lập chế độ Bootloader
  - Bước 2: Kiểm tra điều kiện (Check)
  - Bước 3: Điều hướng luồng (Dispatch)
- Các loại nhân xử lý phổ biến trên thị trường hện nay
  - Nhân ARM Cortex-M, nhân AVR, nhân Xtensa và RISC-V, nhân 8051 cũ và hiện đại, …

# 1.2 – Cơ chế hoạt động

- Các loại nhân xử lý phổ biến

| Họ nhân | Các dòng nhân tiêu biểu | Các chip đại diện |
|---|---|---|
| **ARM Cortex-M** | M0, M0+, M3, M4, M7, M23, M33 | **STM32**, **nRF52832**, Tiva C, SAMD |
| **AVR** | AVR 8-bit, AVR 32-bit, AVR Dx | **ATmega328P** (Arduino), ATtiny, ATmega2560 |
| **RISC-V** | RV32IMAC, RV32E | **ESP32-C3**, GD32VF103, CH32V |
| **Xtensa** | L106, LX6, LX7 | **ESP32**, **ESP8266** |
| **8051 (MCS-51)** | Standard 8051, Enhanced 8051 | STC8, Silicon Labs C8051, N76E003 |
| **PIC / dsPIC** | PIC16, PIC18, PIC24, dsPIC33 | PIC16F877A, dsPIC33 |
| **MSP430** | MSP430X (16-bit) | MSP430G2553, MSP430FR series |
| **MIPS** | MIPS32 M4K, microAptiv | **PIC32** |
| **RL78 / RX** | RL78 (16-bit), RX (32-bit) | Renesas RL78, RX600 |

# 1.2 – Cơ chế hoạt động

- Bước 1: Thiết lập chế độ Bootloader
  - Với các dòng cần **Fuse bits** như AVR
    - AVR sử dụng các ô nhớ đặc biệt gọi là Fuse bits để "cắm mốc" ranh giới cho Bootloader
    - Các bit như BOOTSZ0, BOOTSZ1 quy định kích thước, bit BOOTRST đưa chip vào chế độ Bootloader. Nếu không, mặc định chip sẽ chạy ứng dụng của người lập trình
  - Với các dòng **không cần** Fuse bits
    - Người lập trình thiết kế dự án code, nạp vào địa chỉ Flash phù hợp cho chế độ Custom Bootloader. Fuse bits ở trên cũng đưa Chip vào chế độ này
    - Thiết lập yêu cầu theo nhà sản xuất cho chế độ System Bootloader (có sẵn theo Chip)

# 1.2 – Cơ chế hoạt động

- Bước 2: Kiểm tra
  - Sau khi vào chế độ Custom Bootloader, để quyết định hướng đi tiếp theo cần quét một tín hiệu kích hoạt (Trigger)
    - **Từ phần cứng**: Một nút nhấn được giữ hoặc một chân tín hiệu bị kéo xuống mức thấp…
    - **Từ phần mềm**: Một giá trị đặc biệt (Flag) được lưu trong RAM hoặc EEPROM…
  - System Bootloader (do nhà sản xuất nạp sẵn vào ROM) giúp người dùng nạp được code vào Flash mà không cần dùng mạch nạp chuyên dụng (như ST-Link, J-Link hay USB Blaster) nên không cần Trigger

# 1.2 – Cơ chế hoạt động

- Bước 3: Điều hướng
  - Chế độ cập nhật (Update Mode)
    - Nếu Trigger thỏa mãn (Custom Bootloader) hoặc chế độ System Bootloader, dừng lại để nhận dữ liệu Firmware mới qua các cổng truyền thông
  - Chế độ chạy ứng dụng (Jump Mode)
    - Nếu không có Trigger, Custom Bootloader thực hiện lệnh "nhảy" để bàn giao quyền kiểm soát hoàn toàn cho chương trình chính (Application)

# 1.2 – Cơ chế hoạt động

- So sánh Bootloader trên các loại nhân

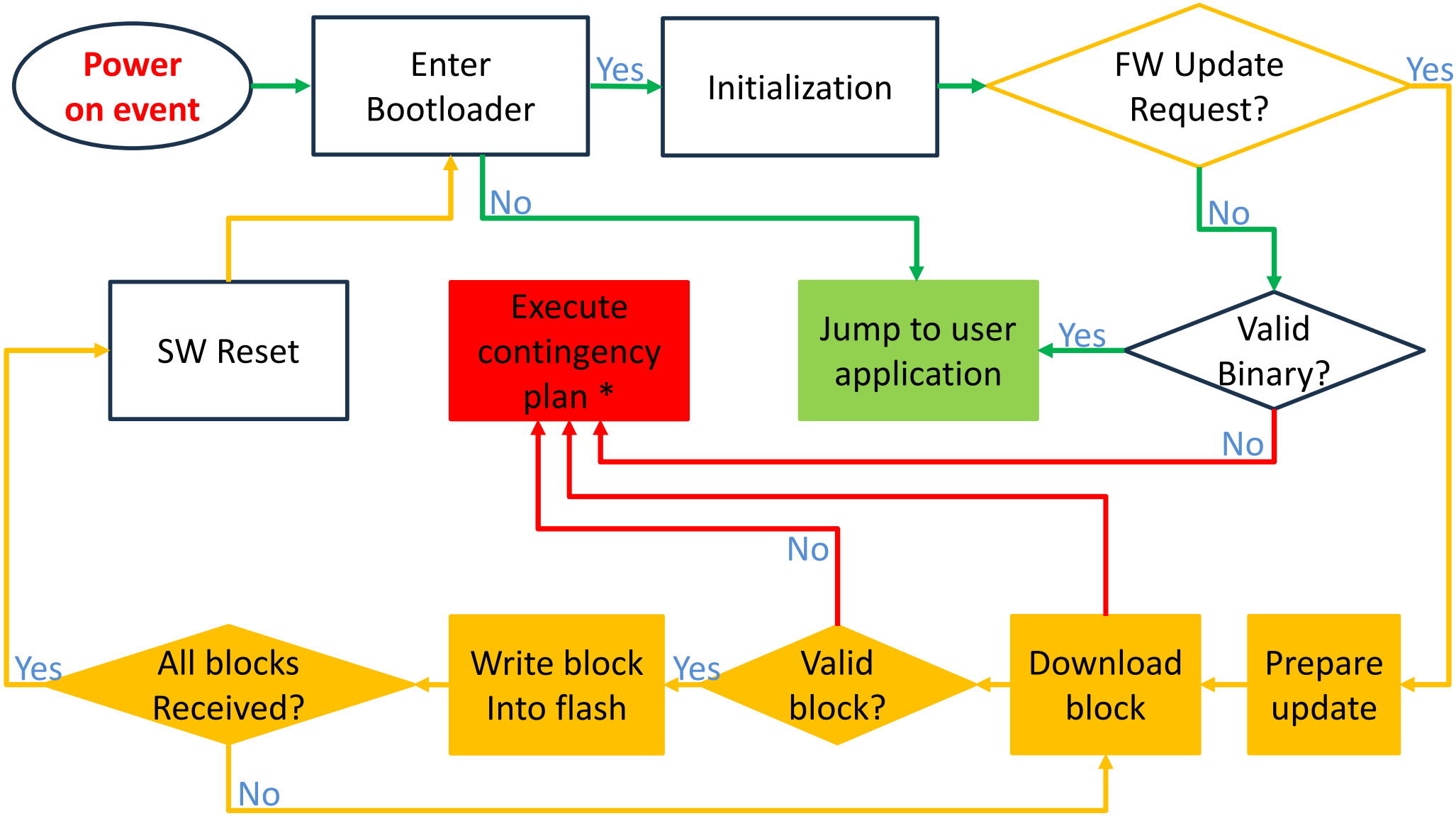
| Nhân xử lý | Có System Bootloader? | Cần thiết lập Fuse/Config? | Có thể Custom Bootloader? |
|---|---|---|---|
| **ARM Cortex-M** | **Có** (Đa số) | Không | **Rất tốt** (Sử dụng VTOR) |
| **AVR** | Không | **Bắt buộc** | Có (Hạn chế về dung lượng) |
| **ESP32 (Xtensa)** | **Có** | Không | Có (Hỗ trợ 2-stage bootloader) |
| **RISC-V** | Tùy hãng (ESP: Có) | Tùy hãng | Có |
| **MSP430** | **Có** (BSL) | Không | Có |
| **8051 (Cũ)** | Không | Không hỗ trợ | Rất khó |

# 1.2 – Cơ chế hoạt động

- So sánh Custom Bootloader AVR và ARM

| Đặc điểm | Dòng AVR | Dòng ARM (STM32, Tiva...) |
|---|---|---|
| **Vị trí Bootloader** | Thường nằm ở **cuối** Flash | Thường nằm ở **đầu** Flash (0x0800 0000) |
| **Cơ chế nhảy** | Dùng **Fuse bits** (BOOTRST) để ép CPU nhảy tới vùng Bootloader | CPU mặc định nhảy vào đầu Flash; việc rẽ nhánh do **Code C** của người lập trình điều khiển |
| **Cấu hình** | Phải dùng mạch nạp chuyên dụng để set Fuse bits một lần duy nhất | Cấu hình linh hoạt thông qua phần mềm (Linker Script và Vector Table) |

# 1.2 – Cơ chế hoạt động




* Kế hoạch dự phòng

# 1.3 – Phân loại

- Có thể chia thành 2 loại
  - System Bootloader
  - Custom Bootloader
- Ngoài ra, 1 trường hợp đặc biệt của Custom Bootloader là In-Application Programming

# 1.3 – Phân loại

- System Bootloader
  - Là một chương trình đặc biệt được nạp sẵn vào chip ngay từ khi sản xuất, không thể thay đổi
  - Được kích hoạt khi thiết lập theo yêu cầu của nsx
  - Vai trò như một "hệ điều hành cứu hộ" nằm trong vùng nhớ ROM
  - Cho phép nạp Firmware mới vào bộ nhớ Flash thông qua các giao diện ngoại vi như UART, USB, CAN, I2C, SPI mà không cần dùng mạch nạp JTAG/SWD chuyên dụng
  - Dòng AVR, 8051 cũ … **KHÔNG** có System Bootloader được nạp sẵn trong ROM giống như các dòng chip 32-bits hiện đại

# 1.3 – Phân loại

- Custom Bootloader
  - Chương trình do chính người lập trình viết và nạp vào vùng Flash thích hợp
  - Tự định nghĩa cách cập nhật rất linh hoạt (ví dụ: lấy code từ thẻ nhớ SD, qua Bluetooth, hoặc mã hóa dữ liệu để bảo mật)
- Custom Bootloader vs System Bootloader
  - Nếu System Bootloader là cứu hộ mặc định của nhà sản xuất, Custom Bootloader chính là điều phối do người lập trình tự thiết kế để điều hành Flash
  - Đây là chương trình chạy ở tầng User Flash, có toàn quyền quyết định việc thực thi hoặc cập nhật ứng dụng mà không cần thiết lập theo yêu cầu từ đầu như System Bootloader

# 1.3 – Phân loại

- In-Application Programming
  - Thực chất là một kỹ thuật nâng cao của Custom Bootloader
  - **Cơ chế**: Chương trình Application đang chạy bình thường vẫn có thể tự nhận dữ liệu mới và ghi vào một vùng Flash khác (thường là vùng tạm). Sau đó chip tự Reset để Bootloader tráo đổi bản cũ và bản mới
  - **Ứng dụng**: Cập nhật từ xa (OTA - Over The Air) cho các thiết bị IoT mà không cần thao tác trực tiếp trên mạch

# 1.4 – Mục đích sử dụng Custom Boot

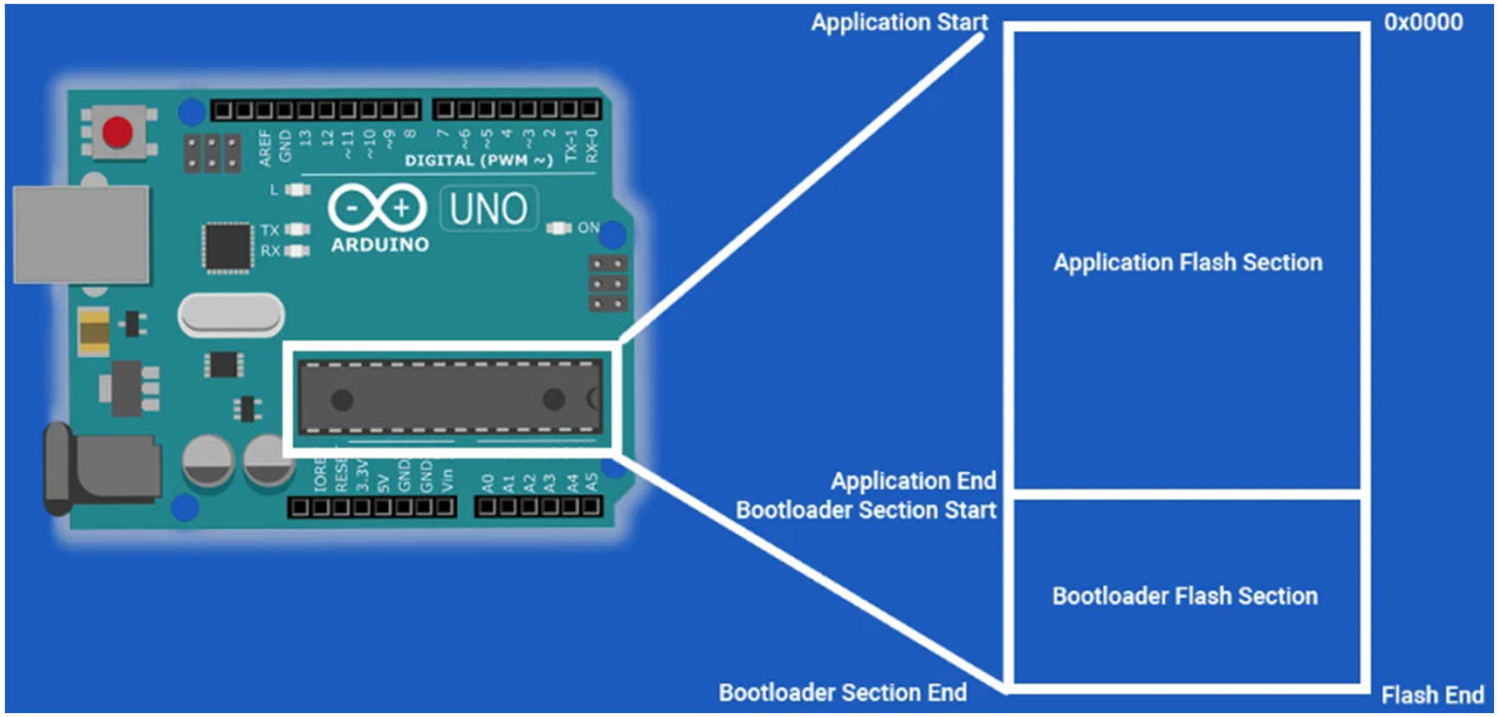
- Cập nhật từ xa/tự động
  - Qua Bluetooth, thẻ nhớ SD, hoặc Wifi thay vì phải cắm dây UART/USB
- Bảo mật (Security)
  - Có thể mã hóa file Firmware. Chỉ Custom Bootloader mới có chìa khóa để giải mã và ghi vào Flash, ngăn chặn việc bị đánh cắp code
- Kiểm tra tính toàn vẹn (Safety)
  - Dùng mã CRC để kiểm tra file nạp có bị lỗi bit nào không trước khi cho phép chạy
- No Brick
  - Xây dựng hệ thống cập nhật Firmware có khả năng dự phòng

# 1.4 – Mục đích sử dụng Custom Boot

| Tiêu chí | Bootloader cơ bản – sơ khởi | Bootloader Dual-Bank – ổn định | Bootloader IoT (OTA) – hiện đại |
|---|---|---|---|
| **Mục tiêu** | Tiện lợi (không cần mạch nạp) | An toàn (chống Brick chip) | Từ xa & Tự động |
| **Rủi ro lỗi nạp** | Chip bị treo (Brick) | Tự động quay về bản cũ (Rollback) | Tự động khôi phục qua mạng |
| **Vùng nhớ** | 1 vùng Application duy nhất | 2 vùng Bank A & Bank B | Phức hợp (Metadata + Multi-Bank) |
| **Ứng dụng** | Học tập, nạp code tại bàn | Thiết bị công nghiệp, y tế | Thiết bị IoT, nhà thông minh |

# 2 – Thiết kế và phân vùng bộ nhớ

- Định nghĩa
- Phân vùng bộ nhớ
- Lưu ý quan trọng



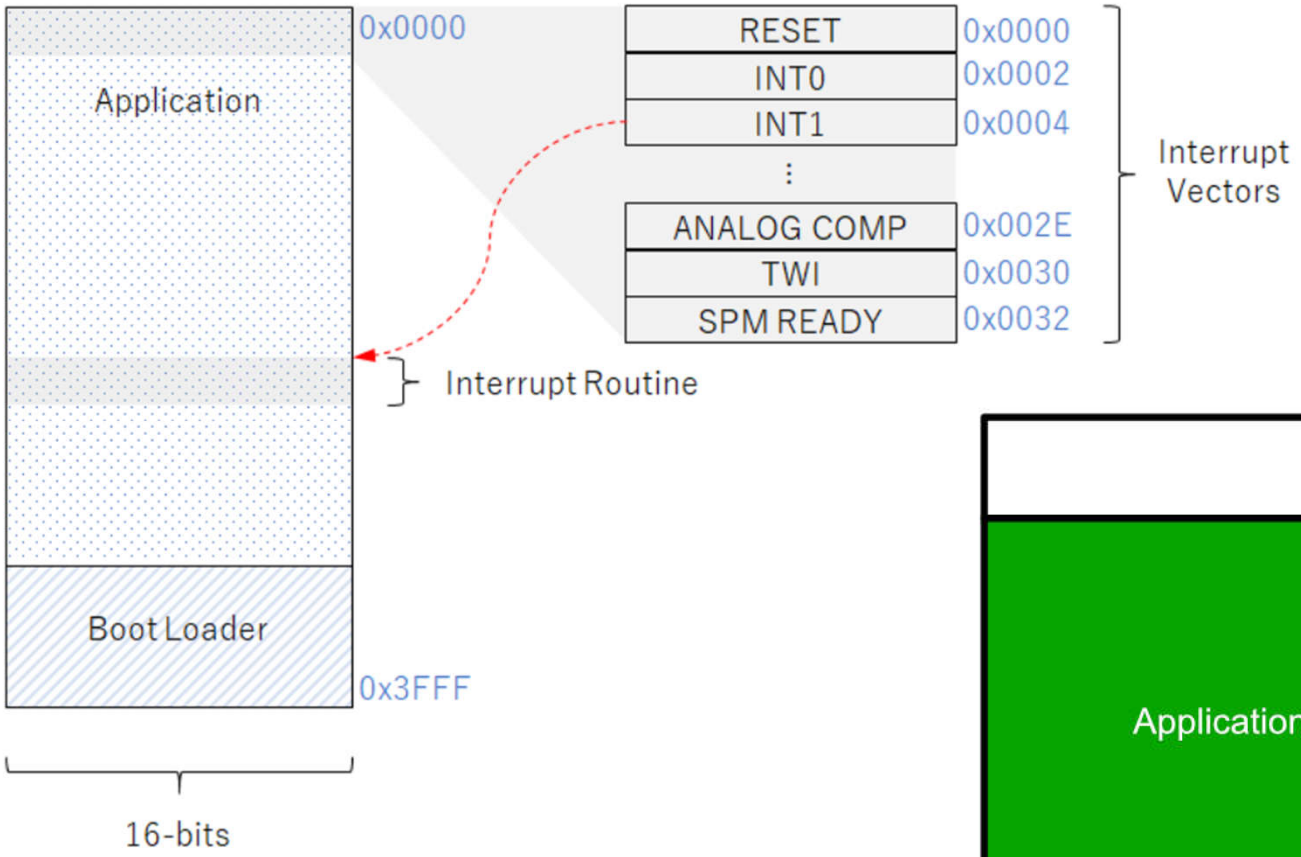
# 2.1 – Định nghĩa

- Phân vùng bộ nhớ là việc chia bộ nhớ Flash vật lý thành các khu vực logic riêng biệt, mỗi vùng đảm nhận một vai trò cụ thể
- Chỉ áp dụng với Custom Bootloader, sau đây gọi tắt là Bootloader
- Mục đích
  - Ngăn chặn việc Bootloader tự xóa chính nó hoặc ghi đè dữ liệu quan trọng, tránh tình trạng "brick" chip
- Tính cố định
  - Các phân vùng này được xác định rõ ràng về địa chỉ bắt đầu và kích thước ngay từ khâu thiết kế hệ thống

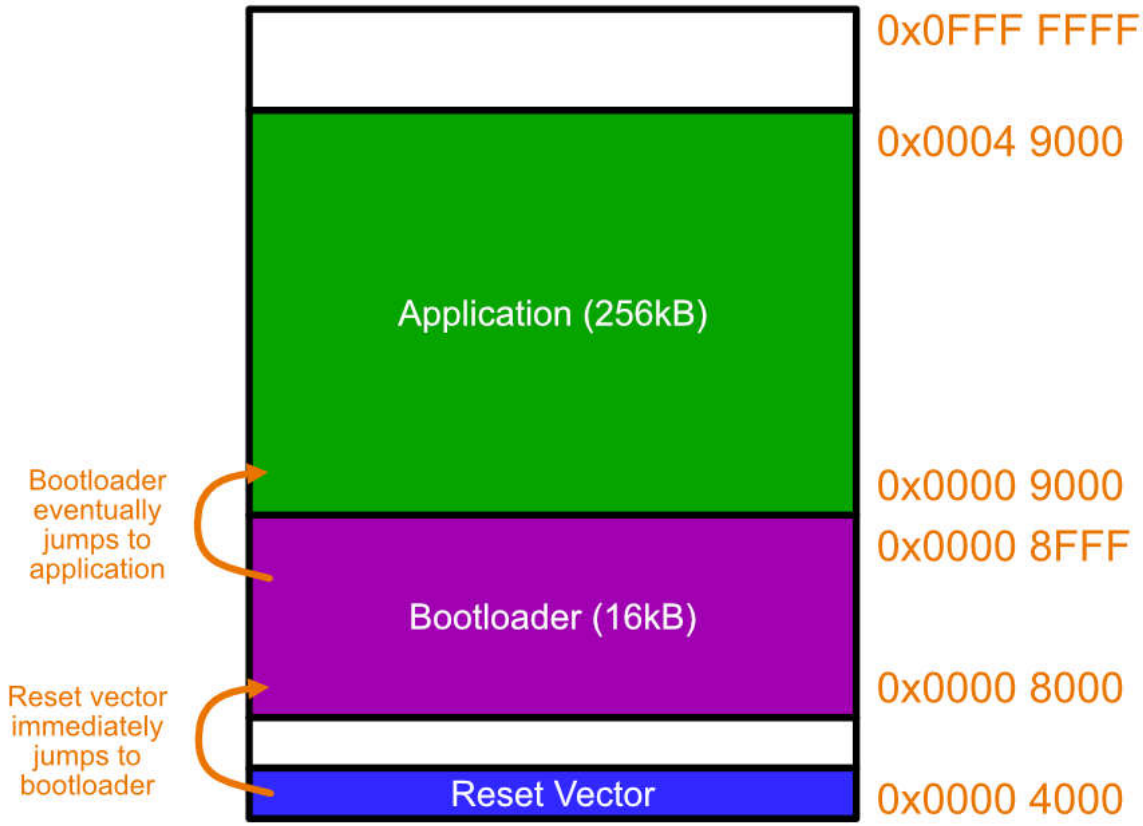
# 2.2 – Phân vùng bộ nhớ

- Cấu trúc vật lý của Flash (Page hay Sector) quyết định trực tiếp chiến lược phân vùng
  - Cấu trúc Page như AVR, STM32F1,…
    - *Đặc điểm*: Toàn bộ Flash được chia thành các trang (Page) có kích thước bằng nhau (thường là 1KB hoặc 2KB)
    - *Phân vùng*: Dễ dàng tính toán, dành ra một số lượng Page nhất định ở đầu hoặc cuối Flash cho Bootloader
  - Cấu trúc Sector như STM32F4,…
    - *Đặc điểm*: Flash chia thành các Sector với kích thước khác nhau (ví dụ: 16KB, 64KB, 128KB, 256KB)
    - *Phân vùng*: Phải thiết kế khéo léo, thường đặt Bootloader vào các Sector nhỏ đầu tiên (như Sector 0 - 16KB) để tránh lãng phí các Sector lớn (128KB, 256KB)

# 2.2 – Phân vùng bộ nhớ




Bản đồ bộ nhớ Flash Chip ATmega328P dùng trong Kit Arduino Uno




Bản đồ bộ nhớ Flash Chip Texas Instruments Tiva C (TM4C123)

# 2.2 – Phân vùng bộ nhớ

- Cơ chế Dual-Bank (Vi điều khiển đời cao)
  - **Đặc điểm**: Flash được chia làm hai nửa vật lý độc lập (Bank 1 và Bank 2)
  - **Phân vùng**: Cho phép thực hiện cơ chế A/B Update: Một Bank chạy ứng dụng cũ, Bank kia nhận dữ liệu mới đồng thời, giúp hệ thống luôn có bản dự phòng an toàn
- Áp dụng Dual-Bank cho Flash chỉ có 1 Bank
  - Với các dòng chip chỉ có 1 khối Flash duy nhất (Single Bank) nhưng dung lượng lớn – chia **Sector**
  - Không thể vừa chạy code vừa ghi vào Flash cùng một lúc trên cùng một chip. Thông thường, Bootloader sẽ phải dừng hẳn việc chạy Application để thực hiện nạp vào Bank trống

# 2.2 – Phân vùng bộ nhớ

| Tiêu chí | Người lập trình tự chia (Logic) | Nhà sản xuất chia sẵn (Hardware) |
|---|---|---|
| **Áp dụng** | Mọi dòng chip có Flash đủ lớn | Chỉ các dòng cao cấp, đắt tiền |
| **Độ an toàn** | Cao (có bản dự phòng để Rollback) | Cực cao (chống Brick tuyệt đối) |
| **Cơ chế nạp** | Phải Reset để cập nhật. Thường nạp trong Bootloader | Có thể cập nhật "nóng" (Live Update) trong khi App đang thực thi |
| **Độ phức tạp** | Thấp, dễ triển khai trên mọi chip | Cao, đòi hỏi phần cứng hỗ trợ chuyên dụng |

# 2.2 – Phân vùng bộ nhớ

- Các thành phần chính
  - Bootloader Region
    - Chứa mã nguồn thực thi của Bootloader, driver ngoại vi (UART, Flash) và logic kiểm tra điều kiện nhảy
  - Metadata / Config
    - Vùng nhỏ chứa thông tin quản lý: Magic Number (xác nhận code hợp lệ), Phiên bản (Version), và mã kiểm tra lỗi (CRC)
  - Application Region
    - Vùng lớn nhất dành cho chương trình chính
      - Single Bank
      - Dual-Bank

# 2.2 – Phân vùng bộ nhớ

- Các thành phần chính

| Thành phần | Dual-bank Tự chia (Logic) | Dual-bank Vật lý (Hardware) |
|---|---|---|
| **Số lượng Bootloader** | 1 (Nằm ở đầu Flash) | 1 (Nằm ở đầu Bank 1) |
| **Vùng Metadata** | 1 vùng chung ở Sector gần Bootloader | 1 vùng chung (thường ở Sector 1 hoặc Option Bytes) |
| **Cách nhảy (Jump)** | Thay đổi địa chỉ con trỏ hàm trong Code | Dùng thanh ghi hệ thống để "tráo đổi" địa chỉ 2 Bank |

# 2.3 – Các lưu ý quan trọng

- Quản lý Bảng Vector ngắt
  - **Vấn đề**: Mặc định bảng Vector nằm ở đầu Flash (0x0800 0000 trên STM32 hoặc 0x0000 trên AVR,…)
  - **Giải pháp**
    - Khi nhảy sang App (ở địa chỉ khác), người lập trình buộc phải cập nhật lại thanh ghi vector (như: `SCB->VTOR` trên ARM) trỏ về địa chỉ bắt đầu của App
    - Nếu không, khi App đang chạy mà có ngắt, CPU sẽ nhảy nhầm về vùng code của Bootloader và gây treo hệ thống
- Dọn dẹp ngoại vi trước khi Nhảy
  - Bootloader thường sử dụng UART, Timer hoặc Clock để phục vụ quá trình nạp
  - Trước khi thực hiện lệnh Jump, phải đưa các ngoại vi này về trạng thái mặc định (`De-init`)

# 2.3 – Các lưu ý quan trọng

- Dọn dẹp ngoại vi trước khi Nhảy
  - **Quan trọng nhất**: Phải vô hiệu hóa ngắt toàn cục (`__disable_irq()` trong ARM) để đảm bảo không có ngắt nào xảy ra trong khoảnh khắc CPU đang chuyển giao quyền điều khiển
- Cấu hình Fuse bits, file Linker Script (.ld), bảng phân vùng (Partition Table) … chính xác
  - File Linker, Partition Table hoặc Fuse bits là nơi định nghĩa "mảnh đất" cho từng chương trình
  - **Bootloader**: Phải giới hạn LENGTH đúng bằng kích thước Sector/Page dự định để tránh code phình to đè vào vùng App
  - **Application**: Phải thay đổi ORIGIN tới địa chỉ bắt đầu mới (sau vùng Bootloader/Metadata)

# 2.3 – Các lưu ý quan trọng

- Kiểm tra tính toàn vẹn
  - Không bao giờ tin tưởng tuyệt đối vào đường truyền UART/OTA… do nhiễu là không thể tránh khỏi
  - **Giải pháp**: Luôn sử dụng Checksum hoặc CRC để kiểm tra file App sau khi nạp xong
  - **Cơ chế**: Chỉ khi mã CRC tính toán khớp với mã lưu trong Metadata, Bootloader mới cho phép thực hiện lệnh Jump

# 3 – Bootloader trên chip STM32F4

- Phân chia vùng nhớ
- Chiến lược phân vùng
- Cấu hình trong Projects

# 3.1 – Phân chia vùng nhớ

- Dòng F405 có cấu trúc Sector với tổng bộ nhớ là 1MB (1024 KB) được phân chia như bảng sau

| Sector | Địa chỉ bắt đầu | Kích thước | Mục đích sử dụng đề xuất |
|---|---|---|---|
| Sector 0 | 0x0800 0000 | 16 KB | Custom Bootloader |
| Sector 1 | 0x0800 4000 | 16 KB | Config Data (Lưu Version, Flag nạp) |
| Sector 2 | 0x0800 8000 | 16 KB | Application Start (Bắt đầu App) |
| Sector 3 | 0x0800 C000 | 16 KB | Application Cont. |
| Sector 4 | 0x0810 0000 | 64 KB | Application Cont. |
| Sector 5-11 | 0x0802 0000 | 128 KB/mỗi | Application Cont. (Vùng trống cực lớn) |

# 3.2 – Chiến lược phân vùng

- Bootloader Region (Sector 0)
  - **Vị trí**: Luôn bắt đầu từ 0x0800 0000
  - **Nhiệm vụ**: Đây là vùng CPU nhảy vào đầu tiên. Nó chứa các driver cơ bản (UART, Flash, GPIO) và logic kiểm tra điều kiện Jump
  - **Lưu ý**: Code Bootloader nên cực kỳ nhỏ gọn (thường dưới 10KB) để vừa vặn trong Sector 0
- Metadata/Signature Region (Sector 1)
  - **Vị trí**: 0x0800 4000
  - **Nhiệm vụ**: Lưu trữ các dấu vết như
    - *Magic Number*: Một con số đặc biệt để xác nhận vùng App đã được nạp code
    - *App Version*: Để Bootloader không nạp lại bản cũ
    - *Checksum (CRC)*: Mã kiểm tra lỗi của toàn bộ vùng App

# 3.2 – Chiến lược phân vùng

- Application Region (Sector 2 đến 11)
  - **Vị trí**: Bắt đầu từ 0x0800 8000
  - **Nhiệm vụ**: Chứa toàn bộ chương trình các tính năng chính của người lập trình
  - **Tại sao bắt đầu từ Sector 2**: Sector 0 và 1 rất nhỏ (16KB), phù hợp làm Bootloader. Từ Sector 5 trở đi kích thước lên tới 128KB, rất lãng phí nếu chỉ để làm Bootloader
  - Cần kiểm soát bộ nhớ nghiêm ngặt

# 3.3 – Cấu hình trong Projects

- Chỉnh sửa file .ld (Linker Script)
  - Đối với Project Bootloader

  Giữ nguyên mặc định vì nó cần nằm ở đầu Flash.

```
/* STM32F405_BOOTLOADER.ld */

MEMORY

{

  FLASH  (rx)          :  ORIGIN  =  0x08000000,  LENGTH  =
16K

  RAM  (xrw)           :  ORIGIN  =  0x20000000,  LENGTH  =
128K

}
```

# 3.3 – Cấu hình trong Projects

- Chỉnh sửa file .ld (Linker Script)
  - Đối với Project Application

Trình biên dịch đặt code từ địa chỉ 0x0800 8000.

```
/* STM32F405_APP.ld */

MEMORY

{

  FLASH  (rx)          : ORIGIN = 0x08008000, LENGTH =
1008K /* 1024K - 16K(BL) */

  RAM  (xrw)           : ORIGIN = 0x20000000, LENGTH =
128K

}
```

# 3.3 – Cấu hình trong Projects

- Chỉnh sửa file main
  - Đối với Project Bootloader
    - Hàm nhảy từ Bootloader đến Application

```
Tham khảo các dự án
```

    - Vô hiệu hóa ngắt toàn cục trước khi nhảy

```
__disable_irq();
```

    - User code
  - Đối với Project Application
    - Dời bảng Vector

```
SCB->VTOR = 0x08008000;
```

    - Bật ngắt toàn cục

```
__enable_irq();
```

    - User code

# 3.3 – Cấu hình trong Projects

- Kiểm soát bộ nhớ dành cho App tránh lấn sang các vùng nhớ cuối

| Khối (Block) | Tên Sector | Địa chỉ bắt đầu (Base Address) | Kích thước (Size) |
|---|---|---|---|
| | Sector 0 | 0x0800 0000 - 0x0800 3FFF | 16 Kbytes |
| | Sector 1 | 0x0800 4000 - 0x0800 7FFF | 16 Kbytes |
| | Sector 2 | 0x0800 8000 - 0x0800 BFFF | 16 Kbytes |
| | Sector 3 | 0x0800 C000 - 0x0800 FFFF | 16 Kbytes |
| **Main memory** | Sector 4 | 0x0801 0000 - 0x0801 FFFF | 64 Kbytes |
| | Sector 5 | 0x0802 0000 - 0x0803 FFFF | 128 Kbytes |
| | Sector 6 | 0x0804 0000 - 0x0805 FFFF | 128 Kbytes |
| | ... | ... | ... |
| | Sector 11 | 0x080E 0000 - 0x080F FFFF | 128 Kbytes |
| **System memory** | | 0x1FFF 0000 - 0x1FFF 77FF | 30 Kbytes |
| **OTP area** | | 0x1FFF 7800 - 0x1FFF 7A0F | 528 bytes |
| **Option bytes** | | 0x1FFF C000 - 0x1FFF C00F | 16 bytes |

**GV: TS. Đào Việt Hùng**
Khoa Điện tử
Trường Điện – Điện tử
Đại học Bách Khoa Hà Nội
Email: hunget.bk@gmail.com